

Types of Check

Type checking is an essential process in programming where a compiler verifies and enforces constraints on data types within a program. It ensures that the code follows the syntactic and semantic rules of the programming language, including type regulations. Type checking assigns types to values, restricts their usage accordingly, and detects any violations. The compiler's type checker module plays a crucial role in managing and computing data types, correcting errors, and maintaining the integrity of the program.

The compiler needs to verify whether the source program follows the syntactic and semantic conventions. This is done with the help of static checking. Static checking helps in reporting programming errors during compile time. Dynamic checking is done during run time to identify and handle errors as they occur. The various static checking that are done by the compiler are listed as follows:

- **Type Checks:** This check determines and handles if an operator is applied to an incompatible operand. Example: The compiler would prompt an error if array variable is added with function variable.
- **Flow of control check:** This would verify whether the statements that results in a branch are terminated correctly. Example: Break statements.
- **Uniqueness check:** This ensures that an object must be defined exactly once in the situation that is demanded by some programming language. Example: labels in case statements need to be unique in pascal, identifiers need to be unique in programming languages.
- **Name-related checks:** This check whether the same name appears more than once in programming languages that does not support. Example: In Ada, a name should not appear more than once and compiler needs to verify this.

Position of type checker

After understanding the types of static checking that are necessary, the compiler does type checking as against any other type of static checking. Typically type checking is being done after successfully parsing the input sentence. The position of the type checker is given in figure where the syntax tree is used to verify the type checking information and is given later to the intermediate code generator for generating intermediate representation.



Figure: Position of the type checker

Thus, as can be seen from figure the type checker is part of the semantic phase of the compiler. Type checking information is added with the semantic rules. Basic type checking is performed which is further extended to type checking of complex attributes.

Type system

A type system is a set of rules used by a compiler to assign types to program elements (variables, expressions, functions) and check whether operations are valid. A type checker implements a type system. The design of a type checker for a language is based on information about the syntactic constructs in the language, the notion of types, and the rules for assigning types to language constructs. For example: “if both operands of the arithmetic operators of +, - and * are of type integer, then the result is of type integer”.

Type Expression

1. A Type Expression is a mathematical notation used to represent the type of a language construct. It can be a simple base type or a complex structure built using constructors. A basic type is a type expression. Integer, float, type_error all are type expression. Statements will have void as their type which is also a basic type expression.
2. A type constructor applied to a type expression is a derived type expression
 - a. Arrays: If T is a type expression then array (A, T) is a type expression denoting the type of the array with elements of type T and index A
 - b. Products: If T1 and T2 are type expressions then their Cartesian product $T1 \times T2$ is a type expression
 - c. Pointers: If T is a type expression then pointer(T) is a type expression indicating the type pointer to an object of type T
 - d. Functions: Represented as $D \rightarrow R$, where D is the domain (input) type and R is the range (output) type.
3. **Type Names:** Aliases given to type expressions (e.g., typedef in C) are themselves considered type expressions.

Specifications of a simple type checker

A type checker is a part of the compiler that verifies whether a program follows type rules (e.g., you don't add an integer to a string). It is usually implemented using syntax-directed

definitions (SDD/SDT). It is specified as a syntax-directed translation scheme that synthesizes the type of each expression from the types of its subexpressions.

Type checking of Expressions

Expressions are to be compatible to be operated on. Every expression is governed by a data type and this 'type' is an attribute associated with it. For type checking of expressions, we will be having semantic rule that verifies this "type" attribute. Below table summarizes the semantic rule for type checking of expressions.

Production	Semantic rule	Explanation
$E \rightarrow \text{literal}$	$E.type = \text{char}$	A literal (like 'a') is a character, so type is char.
$E \rightarrow \text{num}$	$E.type = \text{integer}$	A number without decimal is an integer.
$E \rightarrow \text{id}$	$E.type = \text{lookup}(\text{id.entry})$	Get the type of variable from symbol table.
$E \rightarrow E1 \text{ mod } E2$	$E.type = \text{if } E1.type == \text{integer and } E2.type == \text{integer then integer else type_error}$	mod works only on integers..
$E \rightarrow E1 + E2$	$E.type = \text{if } (E1.type == \text{int AND } E2.type == \text{int}) \text{ then int else real}$	If both operands are same result is assigned to E, otherwise type_error
$E \rightarrow E1[E2]$	$E.type = \text{if } E2.type == \text{integer and } E1.type == \text{array}(s,t) \text{ then } t \text{ else type_error}$	Array index must be integer, result is element type.
$E \rightarrow E1 \uparrow$	$E.type = \text{if } E1.type = \text{ptr}(t) \text{ then } t \text{ else type_error}$	Dereferencing a pointer gives the value it points to.

Type checking of Statements

Statements can be simple assignment statements, conditional statements, sequence of statements or loops. The attribute of the statements is also type and the value of this is void if the statements are correct. The set of semantic rules to perform type checking of statements is given in Table

Production	Rule	Meaning
$S \rightarrow id := E$	Success if $id.type == E.type$, else type-error.	Assignment requires matching types. Result type is void (no value).
$S \rightarrow \text{if } E \text{ then } S_1$	If E is boolean, result type = type of S_1 ; else error.	Condition must be boolean; statement type propagates from S_1 .
$S \rightarrow \text{while } E \text{ do } S_i$	If E is boolean, result type = type of S_i ; else error.	Loop condition must be boolean.

- type-error indicates a static type mismatch.
- void means the statement produces no value (just an effect).
- Arrays are represented as $\text{array}(s, t)$ where s is the subscript range and t is the element type.
- Pointers are $\text{pointer}(t)$ where t is the target type.
- $\text{lookup}(id.entry)$ assumes the symbol table stores each identifier's type.

Functions will do some processing and computations based on its parameters. The type checking of functions verifies whether the function uses arguments that have been passed and checks if there is a mapping between the arguments and the computations of the function.

Production	Semantic Rule	Explanation
$T \rightarrow T_1 \rightarrow T_2$	$T.type = T_1.type \rightarrow T_2.type$	Represents a function type (mapping). The type of T is "from type of T_1 to type of T_2 ".
$E \rightarrow E_1(E_2)$	$E.type = \text{if } E_2.type = s \text{ and } E_1.type = s \rightarrow t \text{ then } t \text{ else type_error}$	Function call: If E_1 is a function that expects type s and E_2 provides type s, then the call result is type t. Otherwise, error.

Type conversion

Data type conversions need to be carried out to support operations on differing data types. For example an integer variable needs to be interacted with the floating point variable. To support such operations type conversions need to be done. The type conversions can be explicit or implicit. Explicit type conversions are done by the programmer in the code itself. Implicit type conversions which are referred to as coercions are done by the compiler. To support coercions semantic rules, need to be written.

Implicit (coercions) Conversion	Explicit Conversion
The data type is converted into another data type by a compiler.	Here data type is converted into another data type by a programmer using casting operator.
This conversion can only be applied to compatible datatypes.	This conversion can be applied to compatible data types as well as incompatible data types.
There is no need for a casting operator.	Casting operator is needed in order to cast a data type to another data type.
This conversion is done at the compile time.	This takes place during the program design by programmer.
It is also called widening conversion because in this, the destination data type cannot be smaller than the source data type.	It is also called narrowing conversion because in this, the destination data type may be smaller than the source data type.
It is less efficient and less reliable.	It is more efficient and reliable.
Example int a = 5; float b = 2.5; float c = a + b; // int → float automatically	Example int a = 5; float b = 2.5; int c = (int)(a + b); // manually converting result to int

Question: Consider the following grammar for arithmetic expression using an operator 'op' to integer or real numbers.

$E \rightarrow E_1 \text{ op } E_2 \mid \text{num.num} \mid \text{num} \mid \text{id}$

Give the syntax directed definitions to determine the type of expression as when two integers are used in expression, resulting type is integer otherwise real.

Production	Semantic Rule
$E \rightarrow E_1 \text{ op } E_2$	if ($E_1.\text{type} == \text{int}$ and $E_2.\text{type} == \text{int}$) then $E.\text{type} = \text{int}$ else $E.\text{type} := \text{real}$
$E \rightarrow \text{num.num}$	$E.\text{type} = \text{real}$
$E \rightarrow \text{num}$	$E.\text{type} = \text{int}$
$E \rightarrow \text{id}$	$E.\text{type} = \text{lookup}(\text{id.entry})$

- **Mixed-Mode Arithmetic:** The rule for implements a "promotion" logic. If both operands are integers, the result remains an integer. If even one operand is a real number, the entire expression is promoted to real.
- **Terminals:**
 - num.num represents a literal with a decimal point (e.g., 3.14), which is always real.
 - num represents a literal without a decimal point (e.g., 5), which is always an integer.
 - id represents a variable whose type is retrieved from the symbol table using a lookup function.

Aspect	Static Type Checking	Dynamic Type Checking
When it happens	At compile time	At runtime
Error detection	Errors found before execution	Errors found during execution
Performance	Faster execution (no runtime checks)	Slower (checks happen during execution)
Flexibility	Less flexible	More flexible
Type declaration	Usually required	Often not required
Safety	More safe (early error detection)	Less safe (errors may occur while running)
Examples of languages	C, C++, Java	Python, JavaScript